Universidad Central del Ecuador

Ingeniería en sistemas

Arquitectura de software

Ing. Noguera

Arquitectura monolítica

Jonathan Plaza

2026-2026

1. Introducción

En el desarrollo de software, la arquitectura monolítica es un modelo tradicional donde todos los componentes de una aplicación (interfaz de usuario, lógica de negocio y acceso a datos) se combinan en un único programa y se despliegan juntos. Sin embargo, dentro de este mismo modelo existen diferentes formas de estructurar el código interno. El presente informe tiene como objetivo contrastar dos enfoques prácticos desarrollados en ReactJS: un diseño desestructurado de alto acoplamiento (conocido coloquialmente como código "Espagueti") y un diseño basado en la separación de responsabilidades a través de una arquitectura limpia en capas (código "Lasaña").

2. Desarrollo y Marco Práctico

Para evidenciar las diferencias entre ambas arquitecturas, se desarrolló un *Dashboard* interactivo que consume datos externos (PokéAPI) para mostrar un catálogo, realizar búsquedas en tiempo real y desplegar información detallada mediante ventanas modales.

2.1. Caso de Estudio 1: Arquitectura "Espagueti" (Monolito Desestructurado)

En esta implementación, la totalidad del flujo de la aplicación se centralizó en un único archivo base (App.jsx).

- **Características Técnicas:** * El manejo de estado (useState), los efectos secundarios (useEffect) y las peticiones de red (fetch) conviven en el mismo bloque.
    - Las reglas de negocio (ej. la lógica para filtrar y limitar los resultados a 20 registros) se calculan directamente en el ciclo de renderizado.
- **Problemas Identificados:** Este enfoque genera un nivel crítico de acoplamiento. La interfaz gráfica depende directamente de la implementación de la API. Si en un futuro se requiere cambiar el proveedor de datos o reutilizar la lógica de búsqueda en otra vista, se requeriría una refactorización masiva del componente, violando el principio de Responsabilidad Única (SRP).

```
Pokemon-Espagueti
├── node_modules
├── public
├── src
│   ├── App.css        (Todos los estilos mezclados aquí)
│   ├── App.jsx        (Aquí vive la UI, el fetch, los estados y el modal)
│   └── main.jsx
├── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

Estructura del proyecto en Espagueti

```jsx
App.jsx
src > App.jsx > ...

import { useState, useEffect } from 'react';
import './App.css';

export default function App() {
  const [pokemones, setPokemones] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);
  const [busqueda, setBusqueda] = useState('');
  const [pokemonModal, setPokemonModal] = useState(null);

  useEffect(() => {
    fetch('https://pokeapi.co/api/v2/pokemon?limit=2000')
      .then((res) => {
        if (!res.ok) throw new Error('Error al conectar con la PokéAPI');
        return res.json();
      })
      .then((data) => {
        // Extraemos el ID real desde la URL en el mismo bloque
        const dataFormateada = data.results.map((poke) => {
          const partesUrl = poke.url.split('/');
          const id = partesUrl[partesUrl.length - 2];

          return {
            id: id,
            nombre: poke.name,
            imagen: `https://raw.githubusercontent.com/PokeAPI/sprites/m
          };
        });
        setPokemones(dataFormateada);
        setCargando(false);
      })
```

Imagen del código app.jsx

2.2. Caso de Estudio 2: Arquitectura "Lasaña" (Monolito en Capas)

En esta segunda implementación, el sistema mantuvo su naturaleza monolítica en cuanto a despliegue, pero su estructura interna se dividió en tres capas lógicas estrictas, comunicadas de forma unidireccional (de abajo hacia arriba).

1. **Capa de Acceso a Datos (Servicios):** Se creó un módulo (pokemonService.js) encargado exclusivamente de las peticiones HTTP y el formateo de los objetos JSON. Es la única capa que conoce la URL de la API.

2. **Capa de Lógica de Negocio (Custom Hooks):** Se implementó el controlador (usePokemon.js). Esta capa consume el servicio, gestiona la memoria de la aplicación (estados) y procesa las reglas de filtrado. No interactúa con elementos HTML ni de presentación.

3. **Capa de Presentación (Componentes UI):** El archivo PokemonDashboard.jsx quedó liberado de cualquier carga lógica. Su única tarea es recibir un arreglo de datos previamente procesado por la capa inferior y renderizar la interfaz web responsiva y el modal interactivo.

```
Pokemon-Lasana
├── node_modules
├── public
├── src
│   ├── components                    (Capa de Presentación)
│   │   ├── PokemonDashboard.jsx      (Solo renderiza el HTML y el Modal)
│   │   └── PokemonDashboard.css      (Estilos exclusivos del Dashboard)
│   ├── hooks                         (Capa Lógica de Negocio)
│   │   └── usePokemon.js             (Maneja el estado y filtra los 20 re
│   ├── services                      (Capa de Acceso a Datos)
│   │   └── pokemonService.js         (Hace el fetch a la PokéAPI)
│   ├── App.css
│   ├── App.jsx                       (Solo invoca a <PokemonDashboard />)
│   └── main.jsx
├── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

Estructura del proyecto en monolítico por capas

```
lasana > src > App.jsx > App
1   import PokemonDashboard from './components/PokemonDashboard';
2
3   function App() {
4     return (
5       <div className="App">
6         <PokemonDashboard />
7       </div>
8     );
9   }
10
11  export default App;
12
```

Imagen Comparativa con app.jsx de lasaña

```
pokemonDashboard.jsx ×
lasana > src > components > pokemonDashboard.jsx > PokemonDashboard
1
2   import { usePokemon } from '../hooks/usePokemon';
3   import './PokemonDashboard.css';
4
5   export default function PokemonDashboard() {
6
7     const {
8       pokemonesFiltrados, cargando, error,
9       busqueda, setBusqueda,
10      pokemonModal, abrirModal, cerrarModal
11    } = usePokemon();
12
13    if (cargando) return <h2>Cargando Pokédex...</h2>;
14    if (error) return <h2>Error: {error}</h2>;
15
16    return (
17      <div className="dashboard-container">
18        <h1>Pokédex (Lasaña)</h1>
19
20        <input
21          type="text"
22          placeholder="Buscar cualquier Pokémon..."
23          value={busqueda}
24          onChange={(e) => setBusqueda(e.target.value)}
25          className="buscador"
26        />
27
28        <div className="pokemon-grid">
29          {pokemonesFiltrados.length === 0 ? (
30            <p>No se encontraron resultados.</p>
31          ) : (
32            pokemonesFiltrados.map((poke) => (
```

Los datos cargados mediante componentes

3. Análisis Comparativo

Criterio de Evaluación	Arquitectura Espagueti (Desestructurada)	Arquitectura Lasaña (En Capas)
Mantenibilidad	Muy baja. Un cambio en la lógica puede romper la UI.	Alta. Los errores se aíslan en su respectiva capa.
Reusabilidad	Nula. La lógica está atrapada en la vista principal.	Alta. Los servicios y hooks pueden usarse en otras vistas.

Criterio de Evaluación	Arquitectura Espagueti (Desestructurada)	Arquitectura Lasaña (En Capas)
Escalabilidad	Deficiente. Agregar nuevas funciones vuelve el código ilegible.	Excelente. Permite crecer el sistema de forma ordenada.
Curva de Desarrollo	Muy rápida al inicio (ideal para prototipos simples).	Requiere planificación inicial, pero ahorra tiempo a largo plazo.

4. Consideraciones de Despliegue y Calidad del Software

Aunque ambas arquitecturas comparten el mismo modelo de despliegue monolítico (por ejemplo, empaquetadas en un único contenedor de **Docker** para su paso a producción), su comportamiento frente a métricas de calidad y metodologías de Integración y Entrega Continua (CI/CD) es diametralmente opuesto.

- **Análisis Estático y Deuda Técnica:** Al someter el código "Espagueti" a herramientas de análisis estático (como **SonarQube**), este reportará altos índices de *Complejidad Cognitiva* y *Code Smells*, ya que las funciones superan el límite recomendado de responsabilidades. Por el contrario, la arquitectura "Lasaña" mantiene métricas saludables gracias a la modularidad de sus servicios y *hooks*.
- **Facilidad de Pruebas (Testing):** En pipelines automatizados (usando herramientas como **Jenkins** o **GitHub Actions**), la arquitectura en capas permite implementar pruebas unitarias aisladas. Se puede probar la lógica de negocio (usePokemon) inyectando datos simulados (*mocks*), sin necesidad de renderizar la interfaz gráfica ni hacer peticiones HTTP reales. En el código desestructurado, las pruebas unitarias son prácticamente imposibles sin levantar todo el entorno de React.

5. Conclusiones

- La arquitectura monolítica no es inherentemente un mal diseño, siempre y cuando su estructura interna respete principios de ingeniería de software como la separación de intereses (SoC).
- El código "Espagueti" es una deuda técnica inmediata. Aunque funcionalmente el programa cumple los mismos requerimientos visuales, carece de calidad interna, lo que encarece drásticamente su mantenimiento.
- La implementación del patrón en capas ("Lasaña") en React demostró que, al delegar responsabilidades específicas a servicios y controladores (Hooks), se obtiene un código modular, altamente cohesivo y preparado para escalar frente a futuros requerimientos empresariales.